

Forest of Distributed B+Tree Based on Key-Value Store for Big-Set Problem

Thanh Trung Nguyen^{1,2(✉)} and Minh Hieu Nguyen²

¹ Research and Development Department, VNG Corporation,
Hanoi, Vietnam

trungthanhnt@gmail.com, thanhnt@vng.com.vn

² Le Quy Don Technical University, Hanoi, Vietnam

Abstract. In many big-data systems, the amount of data is growing rapidly. Many systems have to store big-sets: the sets with a large number of items. Efficiently storing a large number of big-sets to support high rate updating and querying is a challenging problem in data storage systems. Nowadays, distributed key-value stores play important roles in building large-scale systems with many advantages. They support horizontal scalability, low-latency, high throughput when manipulating small or medium key-value pairs. Unfortunately, when working with big-set data structure, they do not work well and most of them are not scalable with a large number of big sets. In this research, we analyze the difficulty in storing big-sets using key-value stores. An architecture called “*Forest of distributed B⁺Tree*” and algorithms are proposed to build NoSql data store for storing big data structures such as set, dictionary. The big-sets are split into multiple small sets of limited size and stored in key-value stores. A Multi-level meta-data is also proposed and used to reduce the complexity in writing operations of big-sets when using key-value stores from $O(N)$ to $O(\log(N))$. This research can store larger number of items in a set than Cassandra and Google BigTable. Parts of big set in this research is distributed while a row in Google BigTable only has a limited size and must be fit in a server. Experiment results show that proposed system has better read performance than Cassandra. The proposed architecture may potentially be used in various applications such as storage system for data from sensors in the Internet of Things (IoT) systems, commercial transaction storages and social networks.

Keywords: Big set · Forest of distributed B+Tree · Key-value · Big data structure · Storage

1 Introduction

Many data systems have to store a large number of big sets in which each set contains a large number of unique items. Each item has three main logical attributes: identification, sorting index and data. Therefore, a set can be represented as a sorted list of unique elements. Each big-set is identified by a *key*. Additionally,

the sets can be frequently changed and potentially grow to millions or billions items. In distributed environment, beside complexity of in-memory data structure, the complexity of adding or removing item in sets can be estimated by the amount of data transfered and disk I/O. The big-set problem thus is how to build a big-set storage system simultaneously satisfies these requirements: (1) ability to store a large number big-sets, size of a set can be larger than size of main-memory or even larger than size of filesystem in a single server; (2) minimize the complexity of updating the sets; (3) all big-set data are persistent, all writes are durable; (4) low latency, high throughput, frequently accessed items are cached in main memory.

Storing big-sets is an important requirement in many systems such as social networks, e-commerces, massively multi-player online games, data for anomaly detection [16], sensors data for Internet of Things(IoT). Since writing rate to the sets is high and the size of sets gets larger overtime, maintaining high performance, throughput and low latency of the store is difficult. Recently, a number of research has been carried out to enhance the performance of key-value store including SILT [10], our Zing Database [15], Mega-KV [22], Masstree [12], etc. Many open-source systems including Redis [20], LevelDB [7], Kyoto cabinet [8] have been developed that support high performance key-value stores. These works aimed to minimize the memory overhead, latency for key-value store. We also had built a backend storage framework on top of these key-value stores to store data for many high load and data intensity applications. They performed efficiently in systems with small or medium size key-value pairs. However, for large data structure in value such as large string set, large dictionary, using these key-value store directly we did not get high performance and system was not well scalable.

This research is done to efficiently solve problems for storing big data structures such as *Set*, *Dictionary/Map* based on key-value stores. These are main contributions of this research:

- Defining big-set problem based on key-value store and analyzing complexity of read and write operations.
- Propose “Forest of distributed B^+Tree ” based on key-value stores, with multi-level meta-data management system, algorithms for minimize data transfer when updating the big sets.
- Implement a big-set storage system based on key-value stores which has ability for storing large number of big-sets. It can store bigger set than row in Google BigTable and Cassandra due to ability to distribute big-set data(or parts of a row) in distributed key-value store. It is useful for Big Data Management.

The remaining two subsections of this section present our framework to build high performance data storage backend service using key-value stores, then big-set problem and its complexity are presented. After that, the rest of the paper is organized as follow: In Sect. 2 we present related works. Section 3 shows the detail architecture and algorithms of proposed “Forest of distributed B+Tree”. Section 4 describes workloads and benchmark and comparison scenario to verify the advantages of proposed work with some discussion. Finally, Sect. 5 concludes the paper.

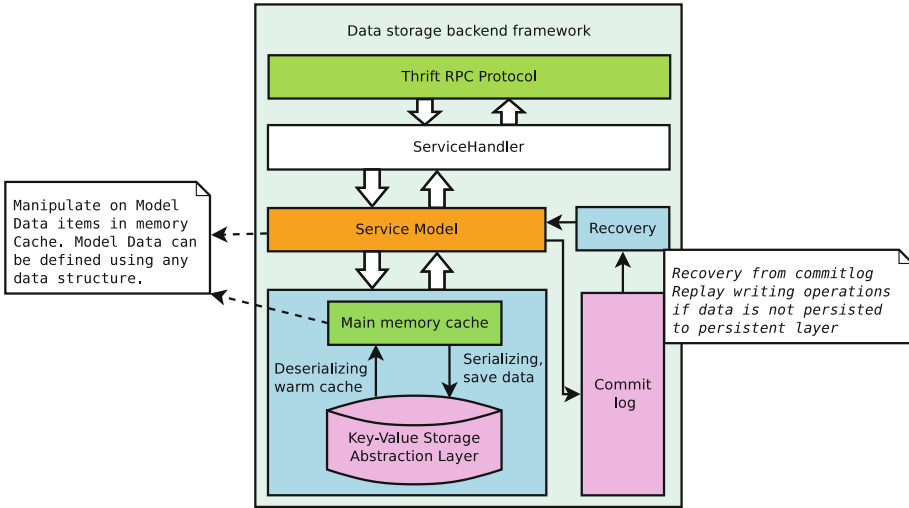


Fig. 1. Storage backend framework overview

1.1 Zing Data Storage Framework

Zing Data Services are used for various products of VNG¹ such as Social Network Media Distribution Platform, Mobile Game Distribution which have millions of Internet users. Zing Data Services require low-latency in every read and write operation. They also require data persistence and fast recovery. We developed a framework for Zing Data Services to satisfy these requirements. This framework is used as a base for building data store service for complex data structures. The framework consists of these components: *Memory Cache*, *Service Model*, *Key-Value Store Abstraction*, and *Commit-Log*. The overview of the framework is described in the Fig. 1. This framework uses key-value stores to persist data. Structured data are serialized and stored in key-value stores. In the read path, if an item is missed from *Memory Cache*, its data will be loaded from key-value store and be de-serialized then be warmed into the *Memory Cache*. Caching data-structures are used to cache structured data as a dictionary, mapping from key to value. Zing data service framework can be used to build high-performance, specific back-end storage system to fit the requirements of applications.

Memory Cache. *Memory Cache* is a dictionary data structure for mapping from keys to structured values. It is used to fast access data with low latency. The size of *Memory Cache* is limited, it only store frequently accessed items. When the number of key-value pairs in *Memory Cache* reaches the configured limited size, some items will be selected and evicted using a replacement algorithm. Each *Memory Cache* class has a replacement algorithm: LRU (least recent

¹ <http://www.vng.com.vn/en/>.

used)[17,18], LFU (least frequent used), ARC (Adaptive Replacement Cache) [13,14], etc. Memory Cache with a specific Cache Replacement Policy is configurable to be appropriate to workload characteristic of the each storage backend service. We implemented hash tables with various cache replacement algorithms for caching data and atomic visiting function to visit item in the cache for reading and/or writing. In this framework, caching functions are thread-safe. Lock-free algorithms can be used to implement some type of cache structures.

Key-Value Store Abstraction. Key-value store abstraction is a persistent layer of the framework. This layer store binary key-value data to persistent devices or remote services. It is abstracted and supports multiple implementation of key-value engines and distributed key-value store. This layer currently support ZDB [15], LevelDB [7], Kyoto Cabinet [8], and Remote Key-Value service wrapped by ZDBService described in [15]. With this design, the framework can support any type of key-value store in the backend. *Memory Cache* and *Key-Value Store Abstraction* are combined to create *Storage* component. This component supports serialization and then saves modified data from memory cache to persistent layer and warm-up cache item when it is missed from cache. The process of flushing modified key-value pairs from *Memory Cache* to *Key-Value Store Abstraction* is controlled by *Storage* component. This process can be configured to be done synchronously or asynchronously.

Service Model. *Service Model* is the core logical layer of the framework. It implements all algorithms and functions for reading, atomic manipulating, modifying, removing model data. Model data in each service is defined depending on the application requirements such as User-Profile, Friend-List, E-Commerce Transaction Info. It can be any type of data structure we want to store. The model uses *Storage* component to read and write data items for its logical algorithms.

Commit-Log. Commit-Log is a component that sequentially logs all write operations to files. All running operations with its parameters are serialized using Thrift binary protocol to get binary data then be appended to commit-log files. Commit-log files are used to recover data after a system crash. All writing to commit-log file is append-only, it is very fast. This component is appropriate with in-memory store service.

1.2 Big Set Problem Statement

Problem. Build key-value store that mapping from key SK_i to value SV_i , each value SV_i is a set of items $\{i_1, i_2, \dots\}$. Item type can be either as simple as integer, double, or as complex as string, binary, a key - value pair or other data structure. There is a function to get the unique sortable key for an item called $K(item)$. For simple item type such as *Integer*, *Double*, $K(item)$ can be the *item* itself.

For key - value item type, $K(item)$ can be equal to $item.key$, etc. The storage system supports these basic functions:

- $add(SK_i, item)$ - add $item$ to value SV_i associated with SK_i .
- $remove(SK_i, itemKey)$ - remove $item$ which have $K(item) = itemKey$ from value SV_i associated with SK_i .
- $existed(SK_i, itemKey)$: boolean - check if there is an item with its key equal to $itemKey$.
- $get(SK_i)$: get all items in the set SK_i .
- $getSlice(SK_i, start, num)$ - retrieve num items in value SV_i from $start^{th}$ item.
- $rangeQuery(SK_i, startItemKey, endItemKey)$ - retrieve items in value SV_i which has key in range of $[startItemKey, endItemKey)$

All data need to be persisted in distributed key-value store.

Complexity. If each set value is stored in a separate data structure such as B-Tree, RB-Tree, the complexity when insert a new item is $O(\log(N))$, N is size of the set. The space complexity and the amount serialized data writing to key-value store is $O(N)$. In case of big set, the big set serialization will make large key-value pairs. And storing these pairs into key-value store when the set is changed by writing operations will consume high the network traffic and disk I/O. That decreases the performance of the whole backend services and increase the load of operation system.

2 Related Works

Many works were done for solving problems in distributed storage system and storing big set data in both academic and industrial fields.

Marcos K. et al. presented a paper about distributed B+Tree [1]. In that work, they used Sinfonia to build distributed B+Tree. In the evaluation the work compared it with Berkeley DB [19] and the design of that work is not for rapidly growing B-Tree so the performance of inserting workload is not very good. Based on [1], a later work [21] proposed an in-memory distributed multi-version B-Tree was proposed. It can scale to hundreds of cores and TBs of memory. It supports copy-on-write snapshot and writable clone.

In [11], Witold Litwin et al. presented a family algorithms called RP* for building distributed files on multicomputers. It proposed distributed data structures and used it for file management in networking environment.

BigTable [3] is a distributed storage system for structured data developed by Google. It is a sparse, distributed, persistent multidimensional sorted map. Data Model of BigTable is indexed by row key, column key, timestamp. Each value in the map is an array of byte. ($row : string, column : string, time : int64 \rightarrow string$ [3]. Data of BigTable is persisted in GFS [5]. BigTable ensures that there is at most one active master at a time by using Chubby [2] -a high

availability and persistent distributed lock service. BigTable partition data of a table into multiple tablets by row keys. Each tablet is a range of a row key. BigTable consists of one master server and many tablet servers. A tablet server manages a set of tablets. The design of BigTable shows that data of a row is stored in a tablet which is assigned to one server at a time. This design leads to the limitation in total size of a row². This research aim to new design that support distribution of partial data of a row into distributed key-value store. Proposed design will allow to store wider row than BigTable.

Cassandra [9] is an open source column-oriented nosql store developed by Facebook³. It support complex data structure in form of columns, each row can consists of multiple columns. Latest version of Cassandra can support up to 2 billions columns per row. Rows in Cassandra belong to a node and can be replicated. This design makes a row in Cassandra never split across nodes and it is impossible to partially cache columns in a row. In production, row cache feature of Cassandra often is disable when using wide row. For Big-Set problem, Cassandra can be used to store big sets of up to 2 billions items. In our benchmarks we have done, Cassandra has good write performance, however, read throughput is lower than the others in the comparison.

Redis [20] is a high performance structured key-value store that supports many data structures. All items in Redis are placed in main memory implemented as a cache with cache-replacement algorithms such as LRU. For storing *Set* data structure, Redis splits items into blocks and stores them in the main memory using skip-list data structure. So, the maximize number of items in a set is limited by the size of main memory and can not be scaled for big-set, each Redis instance can store only a small number of big sets.

3 Forest of Distributed B+Tree for Solving Big-Set Problem

In this section, we show detail idea in our contributions. For storing big set efficiently, we split the big set into multiple limited size small sets and design a multi-level set meta-data to manipulate and query the sets.

We treat a set like a sorted list of unique items. We call l_{max} is the maximum size of each small set. If the size of small set is greater than l_{max} , it will be split into two smaller sets. We designed and implemented a distributed back-end service based on key-value store for storing these small sets. Figure 3 describes the overview of big set components.

Figure 2 shows an example of how big set data are organized. In this example, each small set contains at most 3 items. When the number of items in a small set is greater than 3, it will be split into 2 smaller set. Each item is a key-value pair, sorted in small set by the key. Small sets are stored in *Small Set Service*. There is a multi-level meta-data system to manage the big set. Meta data nodes are stored

² <https://cloud.google.com/bigtable/docs/schema-design>.

³ <https://www.facebook.com>.

in *Metadata Service*. Leaf nodes in Fig. 2 are small sets, the remaining nodes are meta data. *IDGenerator* in Fig. 3 is used to generate Small Set ID, Meta data ID to make keys in *Small Set Service* and *Metadata Service* auto incremental. These services are co-ordinated by *BigSet Service*. That data organization of big set is similar to B+Tree with some modifications as follow:

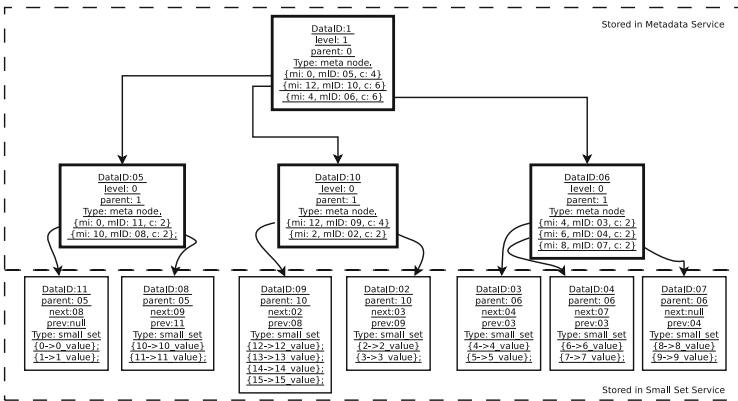


Fig. 2. Big-set sample data for key-value items, each node has at most 4 children

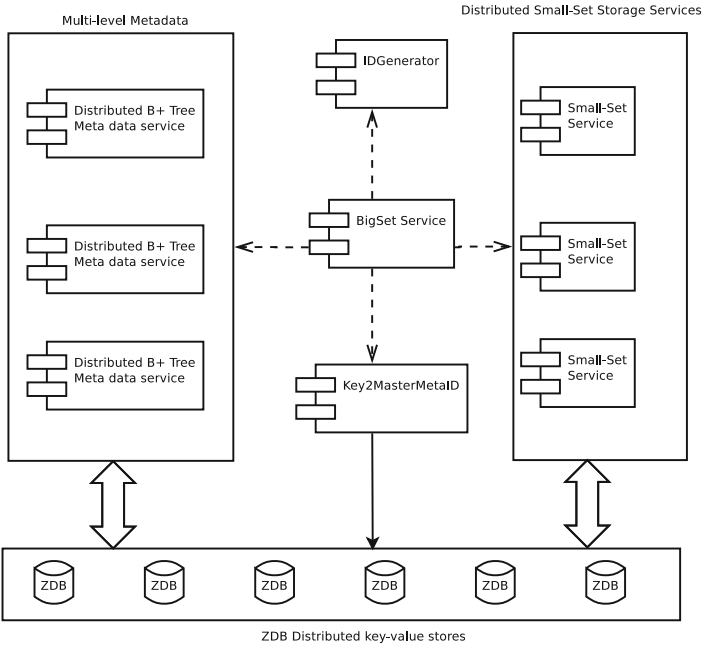


Fig. 3. Big-set components overview

- Tolerate constraints in the number of children in B+Tree nodes. It can be greater than B+Tree defined maximum number for a time before asynchronous splitting operation is completed. And it can be smaller than minimum defined number in B+tree when items are removed from big set.
- Maximum number of items in leaf-node can be different from maximum number of children in non-leaf node.

Each big-set is presented as a Distributed Modified B+Tree where B+Tree nodes are distributed in key-value stores. *BigSet Service* can manage large number of big-sets and called “Forest of Distributed B+Tree”.

3.1 Small Set Service

This service is a key-value store where the key is set id and the value is a set of items. It uses architecture described in Subsect. 1.1. This service supports atomic functions for manipulating small sets. Each set has a limited size and information about adjacent set in the big set. It contains a range of sorted items in the big set. Listing 1.1 below is a sample Thrift type definition in the interface of a Small Set Service.

Listing 1.1. Type definition in a Small Set Service with key-value item

```
struct TItem{
    1: binary key ,
    2: binary value ,
}
typedef binary TItemKey
typedef list <TItem> TItemList
typedef list <TItemKey> TItemKeySet
typedef i64 TSmallSetIDKey
```

The structure of a small set information:

- entries: TItemList - items in the set, this list has a limited size.
- prevSetID: TKey - id of the previous small set in the Big Set. If this field equal to zero, it means that this small set is the first chunk of the Big Set.
- nextSetID: TKey - id of the next small set in the Big Set. If this field equal to zero, it means that this small set is the last chunk of the Big Set.

Small Set Service has functions for query, atomically manipulate data and split when a small set reaches limited size. It consists of these important functions:

- `getSlice(TSmallSetIDKey smallSetID, i32 startIndex, i32 number)`: TItemSet - get a range of *number* items from *startIndexth* item in small set by *smallSetID*
- `getSliceFromItem(TSmallSetIDKey smallSetID, TItemKey startItem, i32 number)` - get up to *number* items in small set, from item *startItem*.

- `addItem(TSmallSetIDKey smallSetID, TItem item)`: `TAddItemResult` - add *item* into small set associate with *smallSetID*. *TAddItemResult* is a return type with following information: error code of operation (EAdded, ENoChange, EReplaced); current size of list after operation; old item if this operation overwrites it by *item* (in case of a complex TItem type).
- `splitSet(TSmallSetIDKey setID, TSmallSetIDKey newSetID, i32 currentSize, TItemKey newSetHead)`: `TSplitResult` - split a small set into two smaller sets.

This service stores frequently accessed small sets in main memory and flushes them into persistent key-value when a set is modified. In the production environment, we deploy multiple instances of ZDBService in many servers for persisting data of small sets. This design of *Small Set Service* is similar to the design of leaf nodes in B+Tree. Each set can be linked to a next small set and a previous small set. Moreover, they are not resided in the only process space, they can be distributed in multiple key-value stores. It is able to travel backward and forward the big set from a specific item or a position in a small set. Figure 2 demonstrates small sets in leaf nodes. In this figure, *DataID* field is the key of key-value store of Small Set Service, *mi*, *mID*, *c* stand for fields: *minItem*, *metaID* and *count* in *TMetaItem* described below.

3.2 Metadata Service

Metadata Service is the most important service in our Big-Set architecture. It stores information about every big-set, and shows how it is organized as multiple small-sets stored in distributed key-value store. This service has algorithms to manipulate metadata and to locate big-set's items. Each Big-Set has a root metadata item, each metadata item contains information about its children and so on till the leaf which stores information about small sets.

Metadata Information. Meta data of a big set is organized as a multi-level structure. Each element is a structure called *TMetaItem* and consists of: *minItem*: lower-bound item of the current branch of B+Tree; *metaID*: identity of meta data (key) in the Metadata Service; *count*: number of item in a branch of B+Tree identified by *metaID*.

Each set's meta-data is described in *TSetMeta* structure. It consists of multiple items of *TMetaItem* type sorted ascending by *minItem* field. *TMetaItem* is the information used to lookup lower level *TSetMeta* or a Small Set stored in Small-Set-Service. *minItem* field is the lower-bound item key of total items managed by the current meta data node and its children. When *level* of a *TSetMeta* is greater than zero, its *children* describe lower level *TSetMeta*, otherwise, they describe small sets. Meta-data Service mapping from *metaID* to *TSetMeta* structure. Each Big-Set is mapped with a root *metaID*. From this root *metaID*, we can travel through whole big-set metadata. The size of the *children* array is limited by a constant called `KMAX_META_SIZE`. Figure 2 demonstrates meta-data in non-leaf nodes.

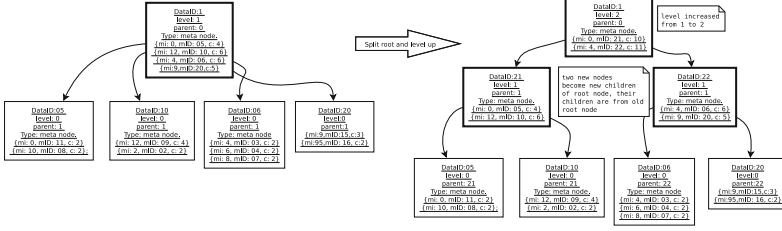


Fig. 4. Split root node and level up

TSetMetaPath is a structure in Metadata Service. It is used to describe a path from a root meta node to a leaf node. It is used in algorithms for querying and manipulating meta data and big-sets as well. It consists of a split info and multiple children called *TSetMetaPathItem*

In *TSetMetaPath* structure, elements of *metaPath* list has decreasing level, the i^{th} element is parent of $i + 1^{th}$ element. The final element in *metaPath* is the parent of *smallSetInfo*.

Metadata Service Algorithms. There are important algorithms in Metadata Service. They are parts of big set operations. The *Traversal* algorithm is used to locate the path from root node to small set associate with an item for adding, modifying or reading. Splitting Algorithms are used to split full Meta-Data node when it reaches the limited number of children. These algorithms are implemented in *Service Model* layer of the framework described Subsect. 1.1.

Traversal Algorithm: To insert an item into a big set, we have to find the small set which will store that item. In this algorithm, a path from root node to leaf node (small set) which may contain a searched itemKey is constructed. At each meta data node, it decides which children node to travel in next step by using binary search in *minItem* field of TMetaItem objects in children of current TSetMeta. And a node with largest number of children will also be updated in *splitInfo* of the meta path. The found small set information will be updated into *smallSetInfo* of TSetMetaPath. Big Set is organized as a distributed B+Tree and the number of node this algorithm will visit is proportional to the height of the B+Tree, so the complexity of this algorithm is $O(\log(N))$, while N is number of items in the Big Set.

Split Root Metadata Node: When a root meta node reaches the limited maximum size, it will be split into two smaller nodes. Two new smaller metadata nodes will be created and the original root node will have two meta items and its level will be increased by 1. Figure 4 demonstrates an example of splitting root meta data node with KMAX_META.SIZE=3.

Split Middle Meta-data Node: This algorithm is used when children number of a non-root meta-data node is greater than limited maximum size caused by

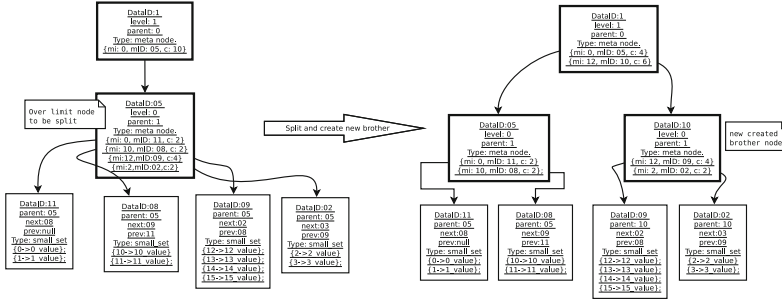


Fig. 5. Split and create new brother data node

smaller level node splitting or small set splitting. The algorithm will create a new same-level metadata node (new brother node), moving second-half children into new node and add new meta item that described new node into the parent meta node. This algorithm is demonstrated in Fig. 5 with $KMAX_META_SIZE=3$.

3.3 BigSet Service

This service is a server application and cares about big set functions for clients. It coordinates other services and uses algorithms from *Small Set Service*, *Metadata Service*. All bigset operations are implemented in this services. We use ZooKeeper [6] to co-ordinate configuration of *Small Set Service* and *Metadata Service* as long as their persistent key-value stores. Each Big Set is organized as a distributed B+Tree. Total Big-Sets managed by *BigSet Service* can be viewed as a “Forest of Distributed B+Tree”.

Add an Item to a Big Set. To add item into a big set, it uses the *Traversal Algorithm* of *Metadata Service* to fill data to an object of class *TSetMetaPath*. The result describes a path from root meta data node to associated small set that will contain adding item.

Item will be added to the found small set from the path. If this is a new item in that small set, the *count* field in associated children of meta data node in the path will be increased by one (or decreased by one in *Remove* function). After adding an item into the small set, it may reach the limited size l_{max} or a full meta data node may be found. If the small set reaches the limited size l_{max} , it will be split into two smaller set in function *checkSplit*. A new small set is created with second-half items from original small set. New *small set id* or key in Small Set Service is generated using *IDGenerator* service. This make Small Sets have auto increasing integer key, that can take advantage of ZDB [15] efficiently when persistently storing data. The Fig. 6 shows an example of splitting small set. This operation always ensures that items in next small set is greater than items in current small set, so we can easily travel backward and forward through big set from a specific position in a small set. This is a characteristic of B+Tree and we can do it on key-value store easily.

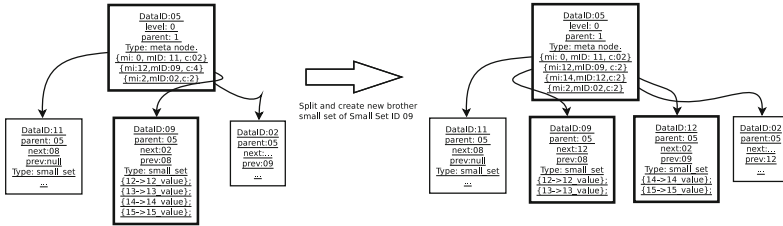


Fig. 6. Split small set example

In case of full a metadata node found in the insertion path, that node in *Metadata Service* will be split using *Split Root Metadata node and Level Up Algorithm* or Algorithm presented in Fig. 5 depend on if that metadata node is root or not. ID of new metadata node or key in *Metadata service* is also generated using *IDGenerator*. It makes the keys of *Metadata service* auto increasing integer and can also take advantage of ZDB efficiently like *Small Set Service*.

The algorithm for *Remove* function of Big Set service is similar to the algorithm for adding an item into big set. However, it eliminates the *checkSplit* step.

Query Items from Big Set Algorithms. Big Set Service provides several type of query in big set:

- **GetSlice:** Query n items from a specific position s .
- **GetSliceFromItem:** Query n_q items from an item key.
- **RangeQuery:** Query items in a range between two item keys such as $[\text{startItemKey}, \text{endItemKey})$.

We developed two approaches for solving these query types. The first approach is suitable for sequential processing in Big Set. While all items in big set are sorted by item keys and according to the design of *Metadata Service* and *Small Set Service* presented above, we can easily do these query types by specifying the starting small set ID of the result. The first small set ID is got by Traversal Algorithm for *GetSliceFromItem* query and *RangeQuery*.

The second approach is suitable for parallel processing in Big Set. In each query, we firstly estimate all small set IDs using Metadata Service for the query. Each meta data element describes a branch of B+Tree. Because meta data elements in children of *TSetMeta* have fields *minItem* - the lower bound item key it stores and *count* - total number of items it stores. These fields are used to retrieve total small set IDs for processing the query efficiently. Then it is able to query these small set data in parallel in *Small Set Service*.

4 Evaluation

In this section, we present workload design, configuration we used to benchmark and compare the proposed big set with existing popular solutions: Redis [20]

and Cassandra [9]. The main purpose of benchmarks is to verify the efficiency of proposed architecture when facing big set storage problems.

We measured number of operations per second (ops), the scalability of the system. We used method of Yahoo! Cloud Serving Benchmark (YCSB) [4] to generate workloads for the benchmarks. Four workloads are described below:

- **Insertion workload:** all clients generate and add new items into big sets.
- **Read Only workload:** all clients check the existence of items in the big set.
- **Mixed workload R:** 90 % client operations read items from big set, 10 % client operations update new value to big set.
- **Mixed workload W:** 10 % client operations read items from big set, 90 % client operations update new value to big set.

In these benchmark workloads, we used 8 storage servers and 4 machines to run clients with configuration below: Ubuntu Server 64 bits, CPU Intel Xeon Quad core, 64 GB DDRam, 600 GB SSD ext4 filesystem, 1 Gbps Network.

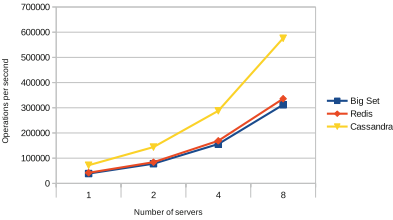


Fig. 7. Write only throughput

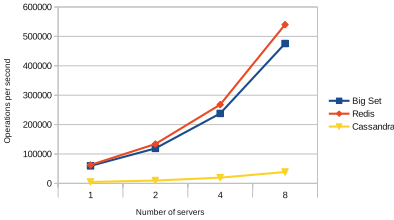


Fig. 8. Read only throughput

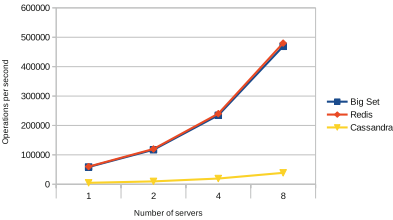


Fig. 9. Mixed Workload R

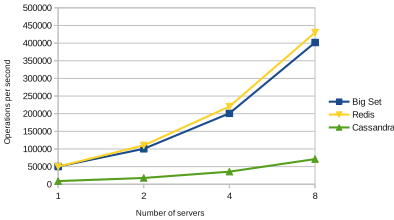


Fig. 10. Mixed Workload W

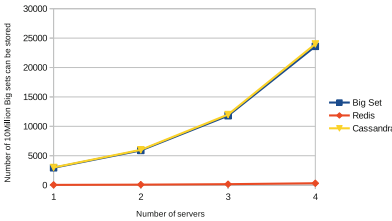


Fig. 11. Capacity comparison

Items in both Cassandra and proposed Big Set for this benchmark are key-value pairs (A simple column in Cassandra is a key-value pair), each big set in Cassandra is a wide row. Items in Redis is a string with the same size to key-value pair in Cassandra.

In client machines, we run 16 processes for each workload. Figure 7 shows the result for write benchmark throughput. Cassandra has the best performance in Write Only Workloads. Proposed Big Set has a bit lower performance compared to Redis Ordered Sets. All these storage system has linear scalability when number of servers increasing.

Figure 8 shows the result of read only benchmark throughput. In spite of having a good write performance, number of read operations per second of Cassandra is much lower than Redis and Proposed Big Set. The same result in Mixed workload is showed in Figs. 9 and 10. Proposed Big Set and Redis outperform Cassandra in reading and mixed workloads. Figure 11 shows the capacity comparison.

5 Conclusion

The proposed Big-Set architecture can be viewed as a *Forest of Distributed B+Tree* with many advantages to build various data storage services. Every Big Set as a value in a key-value store is split into multiple small sets. Small Sets are managed in storage backend service using *Memory Cache* for fast accessing and distributed key-value store to persist data. A *Metadata Service* is designed based on key-value store to manage the big set structure as a distributed B+Tree. It supports building scalable data storage system for big data structure. While other works often build key-value store using B+Tree, this research build distributed B+Tree using key-value store from scratch. This work can easily distribute big value such as big set, big dictionary into multiple servers using advantages of distributed key-value store while others such as Cassandra and Redis have not supported this feature yet. Proposed Distributed Modified B+Tree has a relative high performance in both read and write operations and capability to distributively store big data structures. The proposed work is used to store data for friend suggestion in Zing Me Social Network and to store time series of user action for game data anomaly detection system for games of VNG.

Acknowledgment. This research is funded by Research and Development Department of VNG.

References

1. Aguilera, M.K., Golab, W., Shah, M.A.: A practical scalable distributed B-tree. *Proc. VLDB Endowment* **1**(1), 598–609 (2008)
2. Burrows, M.: The chubby lock service for loosely-coupled distributed systems. In: *Proceedings of the 7th Symposium on Operating Systems Design and Implementation*, pp. 335–350. USENIX Association (2006)

3. Chang, F., Dean, J., Ghemawat, S., Hsieh, W.C., Wallach, D.A., Burrows, M., Chandra, T., Fikes, A., Gruber, R.E.: Bigtable: a distributed storage system for structured data. *ACM Trans. Comput. Syst. (TOCS)* **26**(2), 4 (2008)
4. Cooper, B.F., Silberstein, A., Tam, E., Ramakrishnan, R., Sears, R.: Benchmarking cloud serving systems with YCSB. In: *Proceedings of the 1st ACM Symposium on Cloud Computing*, pp. 143–154. ACM (2010)
5. Ghemawat, S., Gobiuff, H., Leung, S.-T.: The Google file system. *ACM SIGOPS Oper. Syst. Rev.* **37**, 29–43 (2003). ACM
6. Hunt, P., Konar, M., Junqueira, F.P., Reed, B.: ZooKeeper: wait-free coordination for internet-scale systems. In: *Proceedings of the 2010 USENIX Conference on USENIX Annual Technical Conference*, vol. 8, p. 11 (2010)
7. Google Inc.: LevelDB - A fast and lightweight key/value database library by Google (2013). <http://code.google.com/p/leveldb>. Accessed on 23 July 2013
8. FAL Labs: Kyoto Cabinet: a straightforward implementation of DBM (2013). <http://fallabs.com/kyotocabinet>. Accessed on 1 May 2013
9. Lakshman, A., Malik, P.: Cassandra: a decentralized structured storage system. *ACM SIGOPS Oper. Syst. Rev.* **44**(2), 35–40 (2010)
10. Lim, H., Fan, B., Andersen, D.G., Kaminsky, M.: SILT: a memory-efficient, high-performance key-value store. In: *Proceedings of the Twenty-Third ACM Symposium on Operating Systems Principles*, pp. 1–13. ACM (2011)
11. Litwin, W., Neimat, M.-A., Schneider, D.: RP*: a family of order preserving scalable distributed data structures. *VLDB* **94**, 12–15 (1994)
12. Mao, Y., Kohler, E., Morris, R.T.: Cache craftiness for fast multicore key-value storage. In: *Proceedings of the 7th ACM European Conference on Computer Systems*, pp. 183–196. ACM (2012)
13. Megiddo, N., Modha, D.S.: ARC: a self-tuning, low overhead replacement cache. In: *FAST*, vol. 3, pp. 115–130 (2003)
14. Megiddo, N., Modha, D.S.: Outperforming LRU with an adaptive replacement cache algorithm. *Computer* **37**(4), 58–65 (2004)
15. Nguyen, T., Nguyen, M.: Zing Database: high-performance key-value store for large-scale storage service. *Vietnam J. Comput. Sci.* **2**(1), 13–23 (2015)
16. Nguyen, T.T., Nguyen, A.T., Nguyen, T.A.H., Vu, L.T., Nguyen, Q.U., Hai, L.D.: Unsupervised anomaly detection in online game. In: *Proceedings of the Sixth International Symposium on Information and Communication Technology, SoICT 2015*, pp. 4–10. ACM, New York (2015)
17. O’neil, E.J., O’neil, P.E., Weikum, G.: The LRU-K page replacement algorithm for database disk buffering. *ACM SIGMOD Rec.* **22**(2), 297–306 (1993)
18. O’neil, E.J., O’Neil, P.E., Weikum, G.: An optimality proof of the LRU-K page replacement algorithm. *J. ACM (JACM)* **46**(1), 92–112 (1999)
19. Oracle: Oracle Berkeley DB 12c: Persistent key value store (2013). <http://www.oracle.com/technetwork/products/berkeleydb>
20. Sanfilippo, S., Noordhuis, P.: Redis. <http://redis.io>. Accessed on 07 June 2013
21. Sowell, B., Golab, W., Shah, M.A.: Minuet: a scalable distributed multiversion B-tree. *Proc. VLDB Endowment* **5**(9), 884–895 (2012)
22. Zhang, K., Wang, K., Yuan, Y., Guo, L., Lee, R., Zhang, X.: Mega-KV: a case for GPUs to maximize the throughput of in-memory key-value stores. *Proc. VLDB Endowment* **8**(11), 1226–1237 (2015)